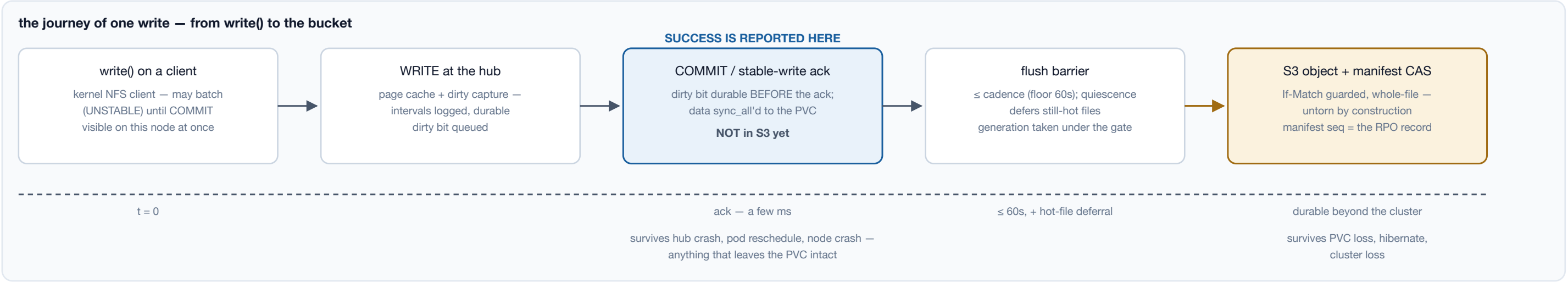


flint-lite is a write-back tier: strong POSIX inside the mount, asynchronous durability to S3. Two statements are both true and must never be conflated — an acked write is crash-durable on the hub's PVC immediately, and it is in S3 only after the next flush barrier. This page is the precise version of both.



who sees your write, and when	
you — the writing client	immediately — your own mount, your own page cache
Pods on the same node	immediately — one kernel mount per node, one shared page cache
a client on another node	at close-to-open — the hub is the one coherence authority: opens, byte-range locks, O_EXCL and rename are real across every client of the share
the project service — S3 browser	after the NEXT FLUSH BARRIER — untorn whole-file generations; the bucket is not a point-in-time snapshot; the manifest (seq) is the closest thing to one
a restarted hub	everything acked — the PVC survives; dirty-bit files re-publish whole-file
after PVC loss	everything PUBLISHED; acked writes inside the RPO window are gone — the manifest counts its own loss bound (beyond_rpo)

not guaranteed — say these out loud

- a successful NFS write is NOT in S3.**
fsync and COMMIT mean the hub's PVC. S3 trails by the flush cadence — that IS the RPO, and it converts to loss exactly when the PVC is lost.
- there is no fsync-to-S3 verb.**
rpoClean (via /status) is the only "everything published" witness — hibernate refuses to delete a PVC without it.
- S3-side writes against a live hub are out of contract.**
three fates: silently destroyed by local-wins, undetected while resident, or adopted as truth by a hydration 412.
- your hottest files trail the most.**
quiescence defers a file written more often than every 10s; on-close is flush-ELIGIBILITY, not flush-now.

S3 itself is strongly consistent (read-after-write 2020-12, conditional writes 2024). The lag is flint's write-back working set — by design.

The contract in one sentence: inside the mount, flint-lite is strong POSIX with the hub as the single authority; toward S3 it is write-back with a stated, measured lag — an acked write is crash-durable on the PVC immediately, and durable in the bucket only after the next flush barrier. The numbers behind the words: flush floor 60s and quiescence window 10s (env-tunable, the share's flush profile), manifest seq + beyond_rpo as the machine-readable RPO record, and the pre-ack durable dirty bit as the reason a hub crash never loses an acked write — a restart re-publishes every bit-set file whole. The one event that converts the RPO into actual loss is PVC destruction, which is why hibernate is verify-then-delete: it will not destroy the PVC until the bucket has provably caught up. "Losing the PVC is a rebuild, not a loss" is therefore true for published bytes and only for published bytes.